



수학 시뮬레이터 제작 보고서

수행평가

[첫 번째 시뮬레이터 제작 보고서]

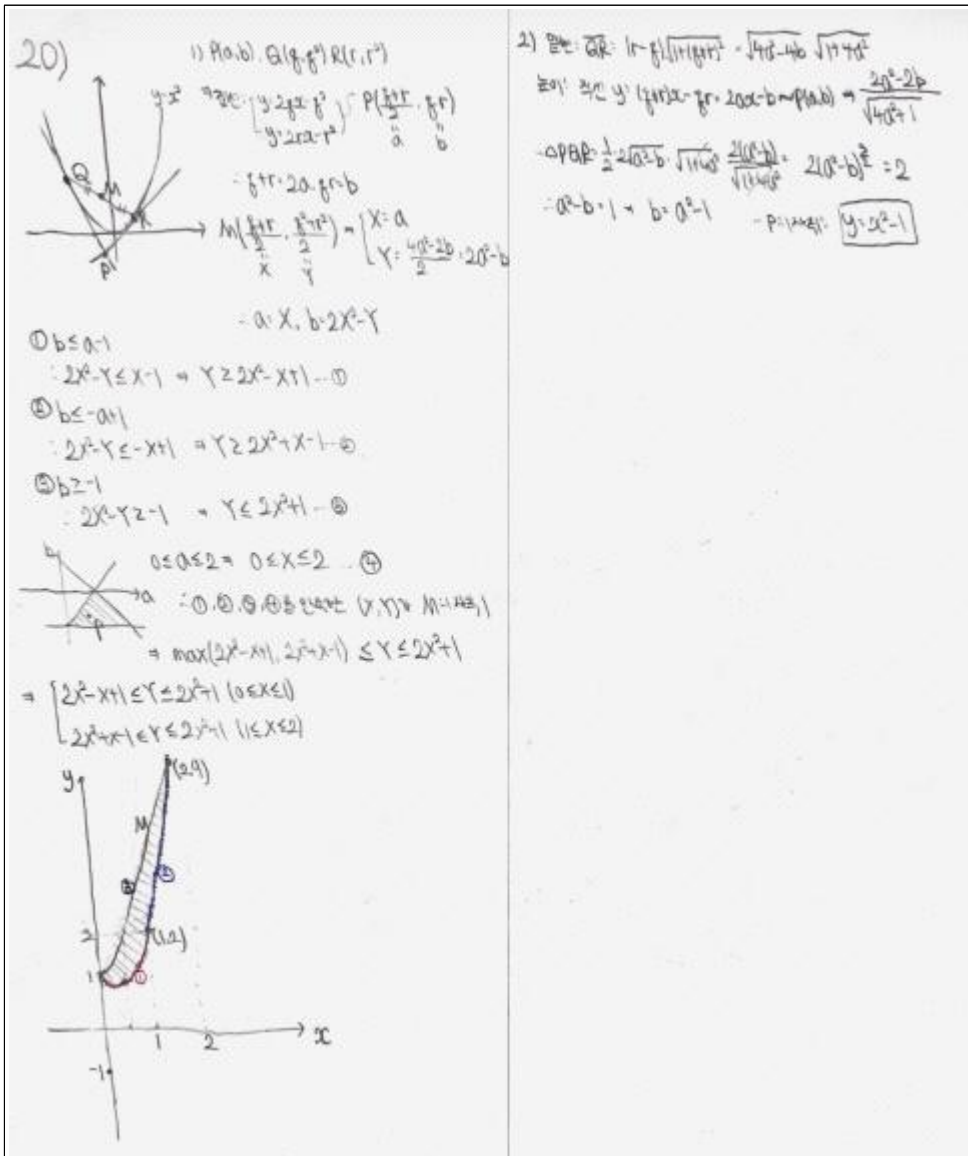
1. 내가 선택한 첫 번째 문제

xy 평면 위의 점 P 에서 포물선 $y=x^2$ 에 두 개의 서로 다른 접선을 그을 때, 그 접점을 각각 Q, R 이라 하자.

(1) 점 P 가 세 부등식 $y \leq x-1$, $y \leq -x+1$, $y \geq -1$ 를 동시에 만족하는 범위에서 움직일 때, 선분 $\overline{QR}$ 의 중점이 움직이는 범위를 도시하라.

(2) $\triangle PQR$ 의 넓이가 2가 되는 점 P 는 어떤 곡선 위에 있는가? 이 곡선의 방정식을 구하라.

2. 이 문제의 풀이



3. 이 문제가 시뮬레이션이 필요하다고 느낀 이유

이 문제는 점 P 가 정해진 영역 안에서 움직일 때, 포물선 $y = x^2$ 에 그은 두 접선의 접점 Q, R 이 함께 변하고, 그에 따라 선분 $\overline{QR}$ 의 중점과 삼각형 $\triangle PQR$ 의 넓이도 동시에 변하는 문제이다. 이 문제는 한 점의 위치 변화가 두 접점과 접선, 중점, 넓이 조건까지 동시에 영향을 주기 때문에 움직이지 않는 그림만으로는 전체 구조를 직관적으로 파악하기 어렵다고 생각했다. 먼저 문제 상황을 정확히 이해하기 위해 시뮬레이션이 필요하다. 점 P 에서 포물선에 그은 두 접선의 접점 Q, R 은 P 의 위치에 따라 계속 달라진다. 이때 P 가 문제에서 주어진 부등식 영역 안에서 움직이면, 접점 Q, R 이 포물선 위에서 어떻게 이동하고, 선분 $\overline{QR}$ 의 중점이 어떤 영역을 채우는지 머릿속만으로 직관적으로 예상하기 어렵다. 시뮬레이터를 통해 점 P 를 직접 움직여 보면 접선 두 개가 생기는 방식과 중점의 이동 범위를 한눈에 확인할 수 있어 문제의 구조를 훨씬 명확하게 파악할 수 있다. 또한, 1번 문항의 자취 영역을 구하는 과정에서 시각적인 힌트를 얻기 위해 필요하다. 이 문제는 P 의 위치에 따라 새롭게 정해지는 선분 $\overline{QR}$ 의 중점이 움직이는 범위를 찾아야 한다. 즉, 원래의 영역이 어떠한 다른 영역으로 변환되는 문제이다. 시뮬레이터에서 P 를 움직일 때 중점의 자취가 실시간으로 그려지도록 하면, 경계선이 어떤 식으로 바뀌는지 관찰할 수 있고, 이를 통해 부등식으로 표현해야 할 영역의 형태를 추측할 수 있다. 그리고 2번 문항에서 $\triangle PQR$ 의 넓이가 2가 되는 점 P 의 위치를 확인하는 데 도움이 된다. 계산 과정에서는 접점의 매개변수를 이용해 넓이 조건을 식으로 정리해야 하지만, 그 결과가 실제로 어떤 곡선을 나타내는지는 그래프를 직접 보기 전에는 직관적으로 알기 어렵다. 시뮬레이터에서 점 P 를 움직일 때 삼각형의 넓이를 실시간으로 표시하고, 넓이가 2가 되는 순간을 강조하면, 계산으로 얻은 곡선의 방정식이 실제 상황과 일치하는지 확인할 수 있다. 따라서 이 문제에서 시뮬레이터는 접선과 접점의 관계를 이해하고, 중점의 자취 영역을 추측하며, 넓이 조건으로 얻은 곡선의 방정식을 확인하는 도구로 활용될 수 있다. 다만 시뮬레이터는 근사적인 시각화를 제공할 뿐이므로, 최종적인 자취 범위와 곡선의 방정식은 접선의 방정식, 근과 계수의 관계, 넓이 계산을 통해 직접 증명해야 한다.

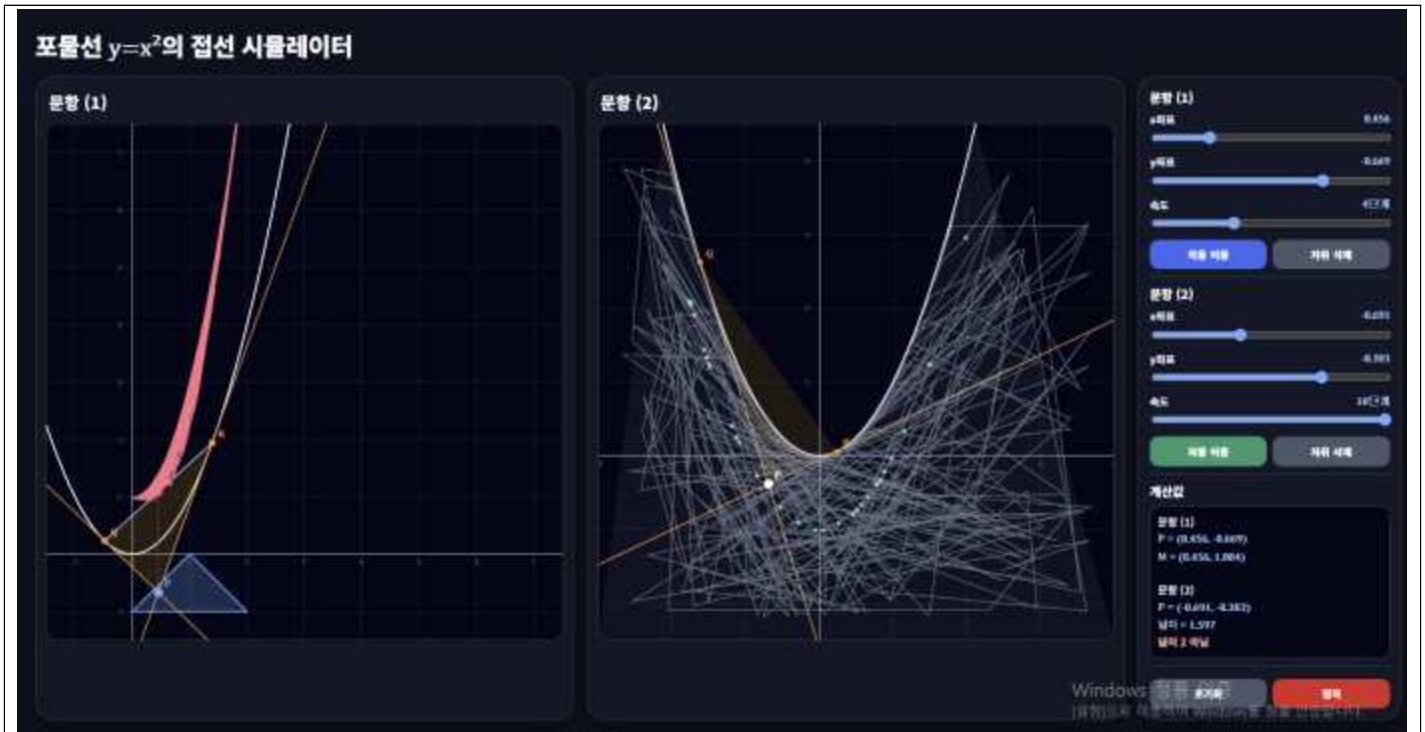
4. 사용한 AI 플랫폼

ChatGPT

5. 시뮬레이터를 만들기 위한 최초의 프롬프트

이 문제를 푸는 데 도움을 줄 수 있는 HTML 시뮬레이터를 만들어줘. 시뮬레이터에는 좌표평면 위에 포물선 $y = x^2$, 점 P , 두 접선, 접점 Q, R , 선분 $\overline{QR}$, 중점 M 이 잘 나타나게 해줘. 또한 1번에서는 점 P 가 움직이는 영역을 음영 표시 해줘. 점 P 는 마우스로 직접 움직이거나 슬라이더로 조절할 수 있고, P 가 자동으로 랜덤하게 움직일 수 있도록 설정할 수 있게 해줘. 이후 P 가 움직일 때 중점 M 의 자취가 실시간으로 그려지며 기록되게 해줘. 또한 P 가 움직일 때 마다 $\triangle PQR$ 의 넓이를 계산해서 소수점 둘째자리까지 화면에 표시하고, 넓이가 2에 가까워질 때는 점 P 의 자취를 다른 색으로 표시해줘. 모든 내용이 한 시뮬레이션 화면에 있으면 복잡할 가능성이 있으니 1번과 2번 문항에 대한 화면을 두 부분으로 나눠줘. 좌표평면과 그래프 화면은 최대한 크게 해주고, 축의 비율 또한 잘 맞게 해줘. 문제의 전체 구조를 직관적으로 예측하기 편하도록 디자인에도 신경써줘, 전체 시뮬레이션 코드는 하나의 HTML 파일로 바로 저장해서 실행할 수 있게 작성해줘.

6. 최초 시뮬레이터의 모습



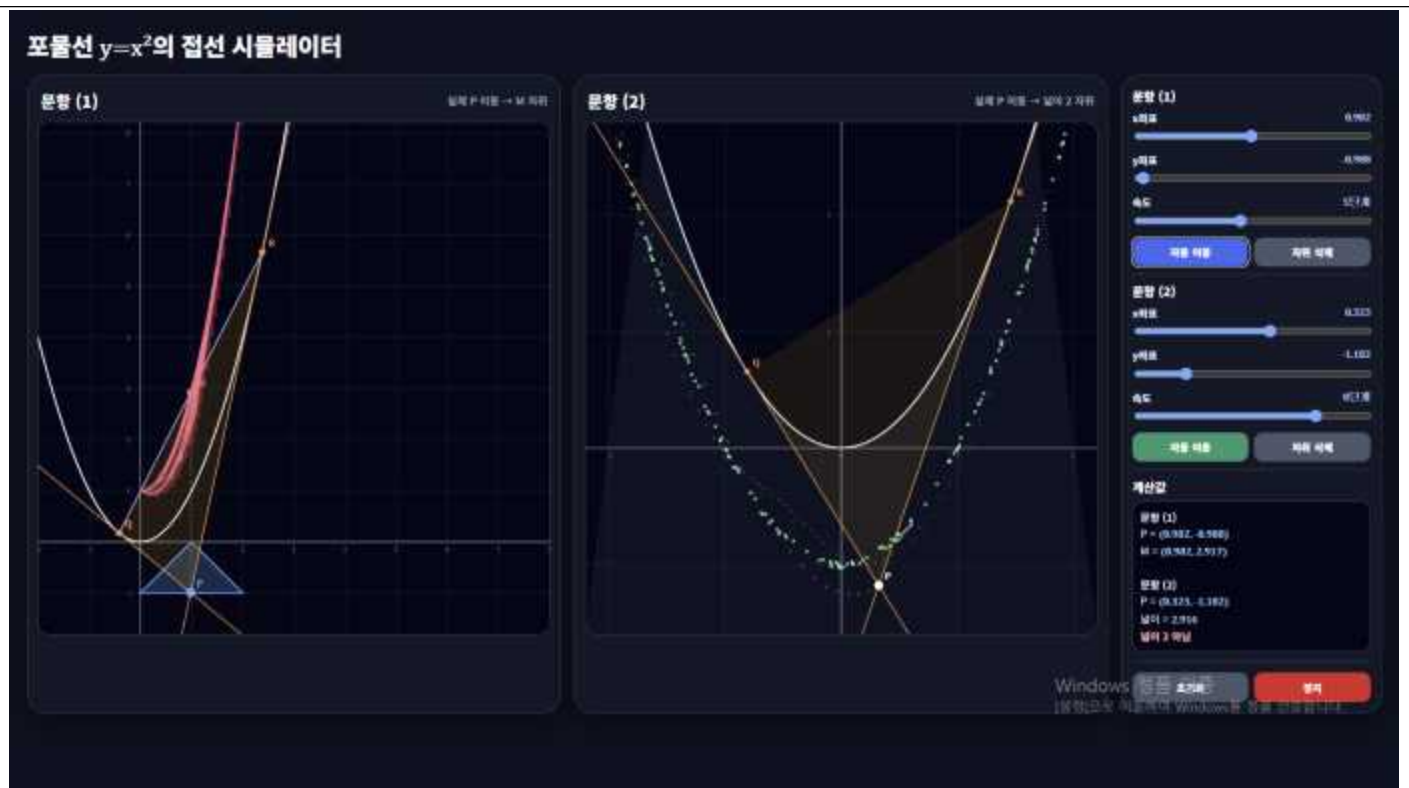
7. 최초 시뮬레이터에서 수정해야 할 사항

최초 시뮬레이터는 포물선 $y=x^2$, 점 P , 접점 Q, R , 중점 M , $\triangle PQR$ 을 한 화면에서 확인할 수 있도록 구성되어 있어 문제 상황을 이해하는 데에는 도움을 주었지만, 점의 움직임과 자취를 통해 문제의 조건을 직관적으로 파악하기에는 몇 가지 수정이 필요했다. 먼저 문항 (1)에서는 점 P 가 움직일 때 중점 M 의 자취가 나타나도록 구성되어 있었지만, 유의미한 영역이 보이기 위해서는 점이 충분히 빠르고 넓게 움직여 자취가 많이 누적되어야 했다. 최초 시뮬레이터에서는 점의 이동 속도와 움직임 방식이 충분하지 않아 M 의 자취가 영역처럼 빠르게 드러나지 않았다. 따라서 자동 이동 방식을 수정하여 점 P 가 주어진 범위 안에서 더 빠르고 고르게 움직이도록 하고, 그에 따라 M 의 자취가 짧은 시간 안에 영역처럼 나타나도록 개선할 필요가 있었다. 또한 문항 (1)과 문항 (2) 모두에서 점이 지나간 자취가 회색 선으로 계속 남는 문제가 있었다. 특히 문항 (2)에서는 점 P 가 넓은 범위를 빠르게 움직이기 때문에 회색 선이 많이 쌓이면서 화면이 복잡하고 산만해졌다. 이로 인해 실제로 관찰해야 하는 넓이 조건의 자취가 잘 보이지 않았다. 따라서 점이 움직이고 있다는 시각적 효과는 유지하되, 지나간 자취가 계속 누적되지 않고 잠시 남았다가 사라지는 형태로 수정할 필요가 있었다. 가장 큰 수정이 필요한 부분은 문항 (2)의 점 P 의 이동 방식이었다. 문항 (2)에서 실제로 넓이 2 조건을 만족하는 P 의 자취는 하나의 곡선 형태로 나타나는데, 최초 시뮬레이터에서는 P 가 매우 넓은 범위를 움직였다. 그 결과 점 P 가 조건을 만족하는 위치 근처를 지나갈 확률이 낮아, 초록색 자취가 찍히는 속도가 매우 느렸다. 따라서 문항 (2)에서는 점 P 의 속도를 문항 (1)보다 더 빠르게 하고, 이동 범위도 실제 정답 곡선에서 크게 벗어나지 않는 주변 영역으로 제한하여 조건을 만족하는 자취가 더 잘 드러나도록 수정할 필요가 있었다. 마지막으로 문항 (1)과 문항 (2)에서 가끔 점들이 제자리에서 빠르게 진동하면서 화면이 지지직거리는 현상이 나타났다. 이는 시뮬레이터를 관찰할 때 불안정하게 보이므로, 점의 자동 이동이 더 부드럽게 이어지도록 수정할 필요가 있었다.

8. 수정하기 위한 프롬프트

지금 문항 (1)에서 M 의 자취가 영역처럼 보이려면 점 P 가 빠르게 움직이면서 자취가 충분히 쌓여야 하는데, 현재는 움직임이 느리고 범위 내에서 이동도 부족해서 M 의 가능 영역이 잘 드러나지 않는 문제점이 있어. 그래서 점 P 가 주어진 영역 안을 더 빠르고 고르게 움직이도록 자동 이동 방식을 수정해줘. P 의 움직임에 따라 M 의 자취가 짧은 시간 안에 충분히 쌓여서 영역처럼 보이게 해줘. 그리고 문항 (1), 문항 (2) 모두에서 점이 지나간 자리에 회색 선이 계속 남는 문제점이 있어. 특히 문항 (2)는 점 P 가 넓은 범위를 빠르게 움직이다 보니까 회색 선이 많이 쌓여서 화면이 복잡하고 산만해져. 그래서 점이 움직이는 느낌은 유지하되, 회색 자취가 계속 남지 않고 잠깐 나타났다가 사라지는 잔상 형태로 바꿔줘. 또 문항 (2)에서는 넓이 2 조건을 만족하는 P 의 자취가 하나의 곡선인데, 현재 P 가 너무 넓은 범위를 움직여서 그 곡선 근처를 지날 확률이 너무 낮아. 그래서 초록색 자취가 찍히는 속도가 너무 느린 문제점이 있어. 문항 (2)의 점 P 는 문항 (1)보다 훨씬 빠르게 움직이게 하고, 이동 범위도 실제 자취 곡선인 $y = x^2 - 1$ 에서 크게 벗어나지 않는 주변 영역으로 제한해줘. 그렇지만 자취 곡선을 처음부터 직접 그리는 말고, 실제로 움직이는 P 가 넓이 2 조건에 가까워질 때만 초록색 점이 찍히게 해줘. 마지막으로 문항 (1), 문항 (2)에서 가끔 점들이 제자리에서 빠르게 진동하면서 지지직거리는 문제점이 있어. 그래서 점의 이동 경로가 더 부드럽게 이어지도록 수정해줘. 점이 한 위치에서 떨리지 않고 자연스럽게 움직이게 해줘.

9. 최종 시뮬레이터의 모습



10. 시뮬레이터 사용 방법

1. 문항 (1) 조작 방법

- 오른쪽 패널의 문항 (1)에서 x 좌표, y 좌표 슬라이더를 움직이면 점 P 의 위치를 직접 조절할 수 있다.
- 점 P 의 위치가 바뀌면 포물선 $y = x^2$ 에 그은 두 접선, 접점 Q, R , 중점 M 이 함께 변한다.
- 자동 이동 버튼을 누르면 점 P 가 주어진 영역 안에서 자동으로 움직이며, 이에 따라 중점 M 의 자취가 왼쪽 화면에 누적된다.
- 자취 삭제 버튼을 누르면 지금까지 생긴 M 의 자취를 지우고 다시 관찰할 수 있다.

2. 문항 (2) 조작 방법

- 오른쪽 패널의 문항 (2)에서 x 좌표, y 좌표 슬라이더를 움직이면 점 P 의 위치를 직접 조절할 수 있다.
- 점 P 의 위치에 따라 접점 Q, R 과 $\triangle PQR$ 의 모양이 변한다.
- 자동 이동 버튼을 누르면 점 P 가 넓이 2 조건이 나타나는 곡선 주변을 자동으로 움직인다.
- $\triangle PQR$ 의 넓이가 2에 가까워지는 순간에는 초록색 점이 찍히며, 이를 통해 넓이 2 조건을 만족하는 P 의 자취를 확인할 수 있다.
- 자취 삭제 버튼을 누르면 문항 (2)에서 찍힌 초록색 자취를 지우고 다시 확인할 수 있다.

3. 계산값 확인

- 오른쪽 아래 계산값 패널에서 현재 점 P 의 좌표, 문항 (1)의 중점 M 의 좌표, 문항 (2)의 $\triangle PQR$ 의 넓이를 확인할 수 있다.
- 넓이가 2에 가까운 경우에는 패널에 상태가 표시되므로, 시각적 자취와 수치 계산을 함께 비교할 수 있다.

11. 시뮬레이터가 자신이 의도한대로 문제를 파악하거나 풀이과정에 대한 힌트를 얻거나 풀이에 대한 검토를 하는 등에 도움이 되었는지, 도움이 되거나 되지 않는 부분에 대해서 구체적으로 서술하세요.

이 시뮬레이터는 점 P 의 위치 변화가 접점 Q, R , 중점 M , $\triangle PQR$ 의 넓이에 어떤 영향을 주는지 확인하는 데 도움이 되었다. 먼저 문항 (1)에서는 점 P 를 직접 움직이거나 자동 이동시켜 보면서, P 가 주어진 영역 안을 움직일 때 중점 M 이 어떤 범위를 채우는지 관찰할 수 있었다. 특히 P 의 위치가 조금만 변해도 접점 Q, R 의 위치와 중점 M 이 함께 변하는 모습이 실시간으로 나타나므로 문제에서 요구하는 영역을 파악하는 데 도움이 되었다. 또한 M 의 자취가 누적되는 모습을 통해, 풀이 과정에서 P 의 좌표와 M 의 좌표 사이의 관계식을 세워야 한다는 힌트를 얻을 수 있었다. 문항 (2)에서는 $\triangle PQR$ 의 넓이가 2에 가까워지는 순간에만 초록색 점이 찍히도록 하여, 넓이 조건을 만족하는 P 의 자취가 곡선 형태로 나타난다는 것을 확인할 수 있었다. 시뮬레이터를 통해 넓이 2라는 조건이 붙으면 가능한 점들이 하나의 곡선 위로 제한된다는 점을 직관적으로 알 수 있었다. 오른쪽 계산값 패널에서 현재 넓이 값을 함께 확인할 수 있었기 때문에, 초록색으로 찍힌 점들이 실제로 넓이 조건과 관련된 점인지 검토하는 데도 도움이 되었다. 하지만 시뮬레이터만으로는 엄밀한 풀이를 완성할 수 없다는 한계가 있다. 화면에 나타나는 자취는 실제 점의 이동과 근사적인 조건 판정을 바탕으로 표시되기 때문에, 정확한 부등식이나 방정식을 직접 증명해 주지는 않는다. 문항 (1)에서 M 의 가능 영역이 화면에 나타나더라도, 그 경계가 왜 그런 식으로 결정되는지는 좌표 관계식을 세워 대수적으로 확인해야 한다. 문항 (2)에서도 초록색 점들이 곡선처럼 보인다는 사실은 힌트가 될 수 있지만, 넓이 조건을 식으로 정리해 실제 자취의 방정식을 구하는 과정은 별도로 필요하다. 따라서 이 시뮬레이터는 정답을 직접 구해 주는 도구라기보다, 풀이 방향을 잡고 계산 결과를 검토하는 보조 도구로 사용하는 것이 적절하다. 한계점을 보완하기 위해서는 시뮬레이터로 먼저 자취의 형태를 예측한 뒤, P, Q, R, M 의 좌표를 문자로 두고 관계식을 세워 정확한 방정식과 부등식을 유도해야 한다. 이후 다시 시뮬레이터의 자취와 비교하면, 계산으로 구한 결과가 실제 문제 상황과 일치하는지 확인할 수 있다.

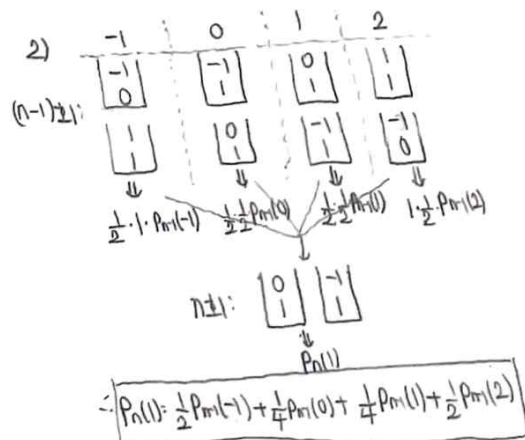
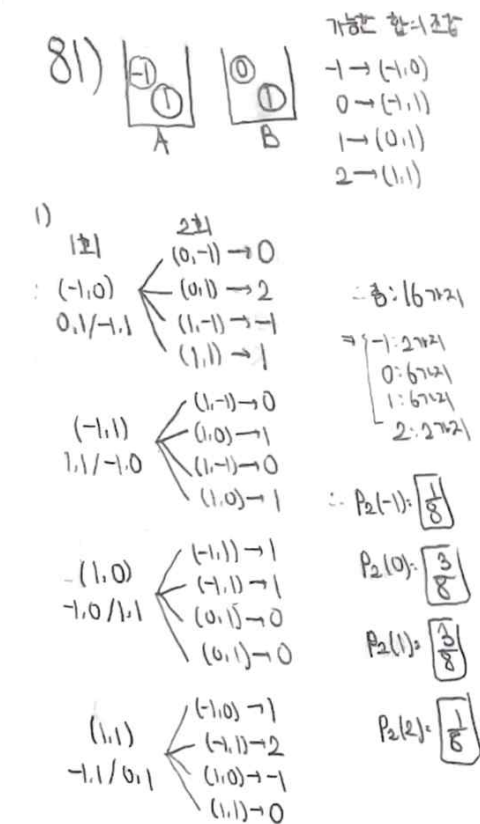
[두 번째 시뮬레이터 제작 보고서]

1. 내가 선택한 두 번째 문제

A 주머니에는 1과 -1의 수가 쓰여진 카드가 각각 1매씩, B 주머니에는 0과 1의 수가 쓰여진 카드가 각각 1매씩 들어 있다. A, B 의 주머니로부터 각각 1장의 카드를 임의로 추출하여 서로 교환하여 주머니에 넣는 시행을 반복하였다. n 회의 시행 후, A 의 주머니 속에 들어 있는 2장의 카드의 수의 합이 k 가 될 확률을 $P_n(k)$ 로 나타낸다. (단, n 은 자연수이다.)

- (1) $P_2(-1), P_2(0), P_2(1)$ 및 $P_2(2)$ 를 각각 구하여라.
- (2) $n \geq 2$ 일 때, $P_n(1)$ 을 $P_{n-1}(-1), P_{n-1}(0), P_{n-1}(1)$ 및 $P_{n-1}(2)$ 를 써서 나타내어라.
- (3) $P_n(1)$ 을 구하여라.

2. 이 문제의 풀이



$$3) P_n(0) = \frac{1}{2}P_{n-1}(1) + \frac{1}{4}P_{n-1}(0) + \frac{1}{4}P_{n-1}(1) + \frac{1}{2}P_{n-1}(2) = P_{n-1}(1)$$

$$P_n(-1) + P_n(2) = 1 - [P_n(0) + P_n(1)] = 1 - 2P_n(1)$$

$$\therefore P_n(1) = \frac{1}{2}[P_{n-1}(-1) + P_{n-1}(2)] + \frac{1}{4}P_{n-1}(0) + \frac{1}{4}P_{n-1}(1)$$

$$= \frac{1}{\pi} [1 - 2P_m(1)] + \frac{1}{2} P_m(1) = \frac{1}{2} - \frac{1}{2} P_m(1)$$

$$P_n = P_{n-1}) \text{ or } 2 \text{ and } \dots \quad P_n = -\frac{1}{2}P_{n-1} + \frac{1}{2} \Rightarrow P_n - \frac{1}{3} = -\frac{1}{2}(P_{n-1} - \frac{1}{3})$$

$$\therefore P_n - \frac{1}{3} = \left(-\frac{1}{2}\right)^{n-2} \left(P_2 - \frac{1}{3}\right) = \left(-\frac{1}{2}\right)^{n-2} \left(\frac{3}{8} - \frac{1}{3}\right) = \frac{1}{24} \left(-\frac{1}{2}\right)^{n-2}$$

$$\Rightarrow P_n = \frac{1}{3} + \frac{1}{6}(-\frac{1}{2})^n$$

$$\therefore P_n(1) = \frac{1}{3} + \frac{1}{6} \left(-\frac{1}{2}\right)^n$$

3. 이 문제가 시뮬레이션이 필요하다고 느낀 이유

이 문제는 A, B 주머니에서 각각 카드 1장씩을 뽑아 서로 교환하는 시행을 반복할 때, A 주머니의 카드 합이 어떤 확률로 변하는지를 분석하는 문제이다. 이 문제에서는 한 번의 시행 결과만 구하는 것이 아니라, 시행이 반복될수록 $-1, 0, 1, 2$ 라는 네 가지 상태 사이에서 확률이 어떻게 이동하는지 그 확률 간의 관계를 파악해야 한다. 따라서 경우의 수를 일일이 구하는 것만으로는 각 상태가 다음 시행에서 어떤 상태로 이어지는지, 그리고 그 확률들이 반복 과정에서 어떻게 누적되는지 직관적으로 이해하기 어렵다고 생각했다. 특히 이 문제에서는 $P_n(1)$ 을 구하기 위해 이전 단계의 확률 $P_{n-1}(-1), P_{n-1}(0), P_{n-1}(1), P_{n-1}(2)$ 가 다음 단계인 n 번째 단계로 어떻게 연결되는지 확인해야 한다. 따라서 이전 상태에서 다음 상태로 이동하는 전이 과정을 점화식으로 표현하여 이웃한 단계 사이 확률을 나타내 풀어야 한다. 시뮬레이터를 만들면 A 주머니와 B 주머니의 카드가 실제로 교환되는 모습을 볼 수 있고, 한 번의 시행 후 A 주머니의 합이 어떤 값으로 바뀌는지 바로 확인할 수 있다. 이를 통해 점화식이 이전 상태에서 현재 상태로 들어오는 확률들을 합한 결과라는 점을 이해하는 데 도움이 된다. 또한 시행 횟수 n 이 커질 때 $P_n(-1), P_n(0), P_n(1), P_n(2)$ 가 어떻게 변하는지 시각적으로 표시하면, 확률 분포가 반복 시행에 따라 변하는 과정을 한눈에 볼 수 있다. 시뮬레이터를 이용하면 실제 실험을 여러 번 반복한 빈도와 점화식으로 계산한 이론 확률을 함께 비교할 수 있다. 따라서 이 문제에서 시뮬레이터는 카드 교환의 규칙을 이해하고, 상태 전이 구조를 파악하며, 점화식으로 구한 확률값이 실제 반복 실험의 경향과 맞는지 검토하는 도구로 활용될 수 있다. 다만 최종적인 $P_n(1)$ 의 일반항은 문제 상황을 바탕으로 점화식을 세우고 이를 대수적으로 풀어야 한다.

4. 사용한 AI 플랫폼

ChatGPT

5. 시뮬레이터를 만들기 위한 최초의 프롬프트

이 문제를 푸는 데 도움을 줄 수 있는 HTML 시뮬레이터를 만들어줘. 문제는 A 주머니에는 1과 -1 카드가 각각 1장씩, B 주머니에는 0과 1 카드가 각각 1장씩 들어 있고, 매 시행마다 A 와 B 에서 카드 1장씩을 뽑아 서로 교환하는 확률 문제야. 먼저, 시행을 반복했을 때 A 주머니 속 카드 2장의 합이 $-1, 0, 1, 2$ 중 어떤 값이 되는지 관찰할 수 있게 만들어줘. 시뮬레이터 화면에는 A 주머니와 B 주머니를 카드 모양으로 시각화해 보여줘. 각 주머니 안에 들어 있는 카드 숫자가 보이게 하고, 1회 시행 버튼을 누르면 A 와 B 에서 카드가 하나씩 선택된 뒤 서로 교환되는 과정이 보이게 해줘. 시행 후에는 현재 A 주머니의 합과 시행 횟수, 마지막으로 교환된 카드가 표시되게 해줘. 자동 시행 버튼과 초기화 버튼도 넣어주고, 여러 번 빠르게 시행할 수 있는 버튼도 만들어줘. 또한 이 문제의 핵심은 상태 전이이므로, A 주머니의 합이 $-1, 0, 1, 2$ 일 때 다음 시행에서 각 상태로 이동할 확률을 전이확률표로 나타내줘. 현재 A 주머니의 합에 해당하는 행은 강조해서, 현재 상태에서 다음 상태로 어떻게 이동할 수 있는지 직관적으로 볼 수 있게 해줘. 오른쪽에는 시행 횟수 n 을 조절하는 슬라이더를 만들고, 점화식으로 계산한 이론 확률 $P_n(-1), P_n(0), P_n(1), P_n(2)$ 를 막대그래프로 보여줘. 그리고 $P_n(1)$ 의 일반항도 화면에 표시해줘. 추가로 반복 실험을 여러 번 실행해서 얻은 실험값과 이론값을 비교할 수 있는 기능도 넣어줘. 전체 시뮬레이터는 하나의 HTML 파일로 바로 저장해서 실행할 수 있게 작성해줘.

6. 최초 시뮬레이터의 모습



7. 최초 시뮬레이터에서 수정해야 할 사항

최초 시뮬레이터는 A 주머니와 B 주머니에서 카드를 뽑아 교환하는 과정 자체는 시각적으로 보여주었지만, 문제의 핵심인 n 회 실행 후 A 주머니의 합이 어떤 확률 분포를 가지는지를 파악하기에는 부족했다. 즉, 우리가 알고자 하는 내용은 여러 번의 실험을 통해 n 번째에 1을 가질 확률을 구하고 싶은데, 현재 시뮬레이터로는 한 번의 실험만이 진행되어 유의미한 결과를 얻을 수 없다. 카드가 한 번씩 교환되는 장면만으로는 $P_n(-1), P_n(0), P_n(1), P_n(2)$ 가 실제 실험에서 어떻게 나타나는지 확인하기 어려웠다. 따라서 한 번의 실험에서 n 번의 카드 교환을 진행하고, 이 실험을 N 번 반복하여 각 결과가 누적되도록 수정할 필요가 있었다. 또한 최초 시뮬레이터에서는 실험 결과와 이론값의 비교가 직관적으로 연결되지 않았다. 실제 상대도수와 이론 확률이 따로 보이거나, 그래프의 의미가 분명하지 않아 문제 풀이와 직접적으로 연결되는 느낌이 약했다. 실험을 통해 얻은 값이 이론과 연결되지 않아, 실험 따로, 이론 따로 느낌이 있었다. 그래서 n 회 실행 후 A의 합이 -1, 0, 1, 2로 나온 횟수와 상대도수를 표와 그래프로 정리하고, 같은 화면에서 현재 n 에 대한 이론값과 비교할 수 있도록 수정해야 했다. 이론 확률 변화 그래프는 실행 횟수에 따른 네 확률의 변화를 보여주는 장점은 있었지만, 이 문제에서 최종적으로 필요한 것은 현재 n 에서의 이론값과 $P_n(k)$ 의 일반항이므로 화면을 복잡하게 만드는 부분이 있었다. 따라서 이론 확률 변화 그래프는 삭제하고, 현재 n 에서의 이론값과 네 가지 일반항만 간단히 표시하는 방식으로 정리할 필요가 있었다. 마지막으로 조작 방식과 화면 구성도 수정이 필요했다. n 과 N 의 의미가 헷갈리지 않도록 n 은 한 실험에서의 카드 교환 횟수, N 은 같은 n 회 실행 실험의 반복 횟수로 구분해야 했다. 또 카드 밑에 불필요한 숫자 표시를 없애고, 카드 중앙에는 숫자만 보이게 하며, 결과만 보기 기능과 교환 속도 조절 기능을 넣어 실험 과정을 천천히 볼 수도 있고 빠르게 결과만 확인할 수도 있도록 개선해야 했다. 이를 통해서 시뮬레이터에 근본적인 목적인, 실험을 통한 이론 값의 검증 및 확률 체감 효과를 개선할 수 있었다.

8. 수정하기 위한 프롬프트

현재 시뮬레이터는 카드가 한 번씩 교환되는 과정 자체에 초점이 맞춰져 있어서, 이 문제의 핵심인 n 회 시행 후 A 주머니의 합이 어떤 확률 분포를 가지는지 확인하기 어려워. 그래서 한 번의 실험에서 n 번의 카드 교환을 진행하고, 같은 n 회 시행 실험을 N 번 반복할 수 있도록 수정해줘. 각 실험이 끝날 때마다 A 주머니의 합이 -1, 0, 1, 2 중 무엇인지 기록하고, 그 결과를 누적해서 횟수와 상대도수로 보여줘. 또 현재 실험 결과와 이론값의 연결이 약하니까, 누적 실험 결과 화면에서 실제 상대도수와 현재 n 에 대한 이론값을 바로 비교할 수 있게 해줘. 막대그래프에서는 실제 상대도수를 표시하고, 이론값은 점선으로 함께 나타내줘. 그리고 아래에는 각 값에 대한 횟수, 상대도수, 이론값을 표로 정리해줘. 이론 확률 변화 그래프는 화면을 현재 문제 풀이에 꼭 필요한 정보는 아니니까 삭제해줘. 대신 현재 n 에서의 $P_n(-1), P_n(0), P_n(1), P_n(2)$ 의 이론값과 네 일반항만 깔끔하게 표시해줘. n 과 N 의 의미도 명확히 구분해줘. n 은 한 번의 실험에서 카드 교환 횟수이고, N 은 같은 n 회 시행 실험을 반복하는 횟수. 이 구분이 입력창, 통계 카드, 설명 문구에 모두 반영되게 해줘. 그리고 실험 상대도수 수렴 그래프에서는 반복 횟수 N 이 커질수록 실제 상대도수가 이론값에 가까워지는 과정을 보여줘. 이때 이론값 점선은 검정색으로 표시해줘. 디자인도 수정해줘. 카드 중앙에는 숫자가 크게 보이게 하고, 카드 교환 애니메이션은 유지하되, 결과만 보기 체크박스를 추가해서 실험 과정을 보다가도 필요하면 남은 실험을 빠르게 처리할 수 있게 해줘. 교환 속도는 실험 도중에도 조절 가능하게 해줘. 코드는 바로 실행할 수 있는 HTML 전체 코드로 줘.

9. 최종 시뮬레이터의 모습

카드 교환 확률 시뮬레이터

n회 시행 반복 실험

실제 반복 실험으로 n회 후 A의 합이 -1, 0, 1, 2가 나오는 상대도수를 직접 확인한다.

시행 횟수 n

12

한 실험에서 카드 교환 횟수이다.

반복 실험 N

16 / 80

같은 n회 시행 실험의 누적 진행이다.

현재 단계

6 / 12

현재 실험 안에서의 교환 진행이다.

현재 A 주머니의 합

1

지금 보이는 실험 상태이다.

마지막 실험 결과

1

n회 후 A 주머니의 합이다.

A 주머니

합 = 1

0

B 주머니

합 = 0

-1

1

17번째 실험 6 / 12번째 교환이다.

n 12

N 80

☐ 결과만 보기

실험 시작

중지

초기화

교환 속도 5

5/10

n: 한 번의 실험에서 카드 교환 횟수이다.

N: 같은 n회 시행 실험을 처음 상태에서 다시 시작해 반복하는 횟수이다.

누적 실험 결과

실선 색상은 실험 상대도수이고, 점선은 현재 n에 대한 이론값이다.

실험 최착별 결과 보기

0.125

0.3125

0.1875

0.375

k = -1

k = 0

k = 1

k = 2

A의 합 k

-1

0

1

2

횟수

2

5

3

6

상대도수

0.125

0.3125

0.1875

0.375

이론값

0.16663

0.33337

0.33337

0.16663

이론값 및 일반항

현재 n에서의 이론값

$P_n(-1)$

0.16663

$P_n(0)$

0.33337

$P_n(1)$

0.33337

$P_n(2)$

0.16663

일반항

k = -1

$P_n(-1) = \frac{1}{6} + \frac{1}{12} \left(-\frac{1}{2}\right)^{n-1}$

k = 1

$P_n(1) = \frac{1}{3} - \frac{1}{12} \left(-\frac{1}{2}\right)^{n-1}$

k = 0

$P_n(0) = \frac{1}{3} - \frac{1}{12} \left(-\frac{1}{2}\right)^{n-1}$

k = 2

$P_n(2) = \frac{1}{6} + \frac{1}{12} \left(-\frac{1}{2}\right)^{n-1}$

실험 상대도수 수렴 그래프

반복 횟수 N이 커질수록 실제 상대도수가 이론값에 가까워지는 과정을 확인한다.



각 실험이 끝날 때마다 상대도수를 누적해 그린 그래프이다. 검은 점선은 현재 n의 이론값이고, 색 실선은 실제 상대도수이다. 반복 실험 횟수 N이 많아질수록 색 실선이 검은 점선에 근처로 모이면, 실험 결과가 이론과 잘 맞는다는 뜻이다.

Windows 정품 인증

(설정)으로 이동하여 Windows를 정품 인증합니다.

10. 시뮬레이터 사용 방법

1. n회 시행 반복 실험

- 조작 방법: n 입력칸에 한 번의 실험에서 카드를 교환할 횟수를 입력하고, N 입력칸에 같은 실험을 반복할 횟수를 입력한다. 이후 실험 시작 버튼을 누르면 실험이 진행된다.
- 기능: 한 번의 실험은 A 주머니와 B 주머니에서 각각 카드 1장을 뽑아 서로 교환하는 과정을 n번 반복하는 방식으로 진행된다. 이 실험을 N번 반복하면서, 각 실험이 끝났을 때 A 주머니의 합이 -1, 0, 1, 2 중 무엇으로 나오는지 기록된다.

2. 카드 교환 과정 관찰

- 조작 방법: 결과만 보기 체크를 해제한 상태에서 실험 시작 버튼을 누른다. 교환 속도 조절 바를 움직이면 카드가 교환되는 속도를 바꿀 수 있다.
- 기능: A 주머니와 B 주머니에서 선택된 카드가 서로 이동하며 교환되는 과정을 직접 볼 수 있다. 이를 통해 한 시행에서 어떤 카드가 바뀌고, 그 결과 A 주머니의 합이 어떻게 변하는지 확인할 수 있다.

3. 결과만 보기 기능

- 조작 방법: 결과만 보기 체크박스를 선택한 뒤 실험 시작 버튼을 누른다. 실험 중간에 체크해도 남은 과정은 빠르게 처리된다.
- 기능: 카드가 하나씩 교환되는 애니메이션을 생략하고, 각 n회 시행 실험의 최종 결과만 빠르게 누적한다. N을 크게 설정하여 실제 상대도수가 이론값에 가까워지는지를 확인할 때 유용하다.

4. 누적 실험 결과 확인

- 조작 방법: 실험이 진행된 뒤 오른쪽의 누적 실험 결과 그래프와 표를 확인한다.
- 기능: 막대그래프는 실제 반복 실험에서 얻은 상대도수이고, 점선은 현재 n에서의 이론값이다. 아래 표에서는 A의 합 k가 -1, 0, 1, 2일 때 각각의 횟수, 상대도수, 이론값을 함께 비교할 수 있다.

5. 이론값 및 일반항 확인

- 조작 방법: 이론값 및 일반항 영역에서 현재 n에 대한 $P_n(-1)$, $P_n(0)$, $P_n(1)$, $P_n(2)$ 의 값을 확인한다.
- 기능: 현재 n에서 각 결과가 나올 이론 확률을 바로 볼 수 있으며, 네 가지 일반항을 통해 확률이 어떤 식으로 계산되는지도 확인할 수 있다.

6. 실험 상대도수 수렴 그래프 확인

- 조작 방법: n 과 N 을 설정한 뒤 실험을 실행하고, 아래의 실험 상대도수 수렴 그래프를 확인한다.
- 기능: 반복 실험 횟수 N 이 증가할수록 실제 상대도수가 어떻게 변하는지 보여준다. 검은 점선은 이론값이고, 색 실선은 실제 상대도수이므로, 실험 결과가 이론 확률에 가까워지는 과정을 직관적으로 확인할 수 있다.

11. 시뮬레이터가 자신이 의도한대로 문제를 파악하거나 풀이과정에 대한 힌트를 얻거나 풀이에 대한 검토를 하는 등에 도움이 되었는지, 도움이 되거나 되지 않는 부분에 대해서 구체적으로 서술하세요.

시뮬레이터는 n 회 시행 후 A 주머니의 합이 어떤 상태로 분포하는지를 확인하는 확률 실험으로 이 문제를 파악하는 데 도움이 되었다. 시뮬레이터에서 한 번의 실험을 n 회 시행으로 묶고, 이를 N 번 반복하여 결과를 누적하니, 문제의 핵심인 n 회 후 A 의 합이 $-1, 0, 1, 2$ 중 어디에 도달하는 확률이 분명히 나타났다. 또한, 풀이 과정에 대한 힌트를 얻는 데에도 도움이 되었다. 실험을 여러 번 반복하면 A 의 합이 $-1, 0, 1, 2$ 의 네 가지 상태로만 나타나므로, 이 문제를 네 상태 사이의 전이 과정으로 볼 수 있다는 생각을 할 수 있다. 특히 $P_n(1)$ 을 구하려면 이전 단계에서 바로 1이 되는 경우만 보는 것이 아니라, $P_{n-1}(-1), P_{n-1}(0), P_{n-1}(1), P_{n-1}(2)$ 각각에서 다음 시행 후 1로 이동할 확률을 모두 고려해야 한다는 점을 직관적으로 이해할 수 있었다. 따라서 시뮬레이터는 점화식을 세우는 방향을 잡는 데 유용했다. 풀이 검토에도 도움이 되었다. 손으로 구한 일반항을 시뮬레이터의 현재 n 에서의 이론값으로 표시하고, 실제 반복 실험의 상대도수와 비교할 수 있도록 하였다. N 을 작게 설정하면 실제 상대도수와 이론값 사이에 차이가 있지만, N 을 크게 하면 색 실선이 검은 점선 근처로 모이는 것을 확인할 수 있다. 이를 통해 계산한 이론값이 실제 무작위 실험과 대체로 일치하는지 검토할 수 있었다. 도움이 되지 않는 부분도 있었다. 시뮬레이터는 실험 결과와 이론값의 관계를 보여주지만, $P_n(1)$ 의 일반항을 대수적으로 유도하는 과정 자체를 대신해 주지는 못한다. 점화식을 세운 뒤 이를 정리하여 일반항을 구하는 계산은 여전히 직접 해야 한다. 또한 N 이 작을 때는 실제 상대도수가 이론값과 꽤 다르게 나올 수 있으므로, 한두 번의 시뮬레이션 결과만 보고 이론값이 틀렸다고 판단해서는 안 된다. 이 한계를 보완하기 위해 시뮬레이터에는 현재 n 에서의 이론값과 네 가지 일반항을 따로 표시하였다. 즉, 시뮬레이터는 직접 세운 점화식과 일반항이 실제 확률 실험의 경향과 맞는지 확인하는 보조 도구로 활용하는 것이 적절하다. 최종적으로 이 시뮬레이터는 문제 상황을 확률 상태 전이로 이해하고, 실험 상대도수와 이론 확률을 비교하며, 풀이 결과를 검토하는 데 효과적으로 도움이 되었다.

[마무리]

1. 두 문제에 대한 시뮬레이터를 기획하고 활용하여 문제를 해결하는 전 과정에서, 생성형 AI가 제공할 수 있는 가치와 반드시 인간(학생 본인)이 주도해야 하는 수학적 사고의 영역을 구분하여 서술하세요.

두 문제의 시뮬레이터를 제작하면서, 생성형 AI는 아이디어를 빠르게 코드로 구현하고, 화면 구성이나 기능을 실제 프로그램 형태로 만들어 주는 데 큰 가치가 있다고 느꼈다. 특히 HTML 코드 작성, 그래프 구현, 버튼과 슬라이더 같은 조작 기능 추가, 반복 실험 결과를 시각화하는 작업은 AI의 도움을 받으면 훨씬 빠르게 진행할 수 있었다. 즉, AI는 내가 생각한 수학적 아이디어를 실제로 움직이는 시뮬레이터로 바꾸어 주는 역할로서 매우 효과적이었다. 그러나 AI가 처음부터 문제의 핵심을 정확히 이해하는 것은 아니었다. 어떤 부분이 수학적으로 중요한지, 어떤 장면이 인간의 입장에서 이해하기 어려운지, 어떤 시각화가 풀이에 직접적인 도움을 주는지는 AI가 스스로 판단하기 어려웠다. 이런 핵심을 정하는 것은 나 본인이 직접 해야 했다. 또한 시뮬레이터가 실제로 수학적 사고를 돕는지, 기능이 문제의 조건과 맞는지, 결과가 이론값과 타당하게 연결되는지는 인간이 계속 검토해야 한다고 생각했다. AI는 코드를 만들 수는 있지만, 그 코드가 문제 풀이에 적절한지, 화면 배치가 보기 편한지, 사용자가 무엇을 보고 이해해야 하는지까지 완전히 판단하지는 못했다. 따라서 생성형 AI는 구현과 수정의 속도를 높여 주는 도구이고, 문제의 핵심 파악, 수학적 구조 설정, 결과 해석, 최종 검토는 학생이 주도해야 하는 영역이라고 생각했다.

2. 복잡한 수학 문제를 AI와 협업하여 시뮬레이터로 구현해 본 경험이, 향후 자신이 희망하는 진로 분야에서 실무적인 문제를 해결하는 데 어떤 시사점을 주는지 서술하세요.

이번 경험은 공학 분야에서 시뮬레이션이 문제를 이해하고 검토하는 중요한 도구가 될 수 있다는 점을 보여 주었다. 실제 공학 문제에서는 모든 조건을 직접 실험하기 어렵거나, 시간과 비용이 많이 드는 경우가 많다. 이때 시뮬레이션을 활용하면 다양한 조건을 빠르게 바꾸어 가며 결과를 예측하고, 실제 실험 전에 핵심 변수를 파악할 수 있다. 특히 내가 관심을 가지는 화학공학 분야에서도 이러한 방식은 중요하다고 생각한다. 예를 들어 반응 조건, 유량, 농도, 온도, 촉매 성능, 배양 조건 등을 모두 실험으로만 확인하려면 많은 시간과 자원이 필요하다. 하지만 AI를 활용해 간단한 모델이나 시뮬레이터를 빠르게 만들 수 있다면, 어떤 변수가 결과에 큰 영향을 주는지 먼저 탐색하고, 실제 실험은 더 효율적으로 설계할 수 있다. 실제로, 학교에서 진행하는 과학 연구에서 또한 실험 중 물리량 측정과 이론 분석을 위해 AI를 이용해 구현한 시뮬레이터를 이용 중이다. 또한 AI는 복잡한 코드를 처음부터 모두 직접 작성해야 하는 부담을 줄여 준다. 중요한 것은 코드를 한 줄씩 직접 짜는 능력만이 아니라, 해결하려는 문제를 정확히 정의하고, 필요한 기능을 구체적으로 요구하며, 결과가 타당한지 검토하는 능력이라고 느꼈다. 앞으로 공학 분야에서도 AI를 활용해 모델을 빠르게 만들고, 시뮬레이션 결과를 바탕으로 실제 실험과 설계를 개선하는 역량이 더욱 중요해질 것이라고 생각했다.

3. 보고서를 작성하며 느낀점을 간단히 작성하세요.

이전에 웹사이트를 만들어 본 경험이 있었기 때문에 HTML 시뮬레이터 제작 자체는 크게 낯설지 않았다. 그 이후로 ChatGPT, Claude, Gemini 등 여러 생성형 AI를 활용해 게임, 시뮬레이터, 간단한 프로그램을 만들어 보는 데 관심이 많아졌고, 이번 수행평가에서도 그 경험이 도움이 되었다. 이번 보고서를 작성하면서 가장 크게 느낀 점은 이제는 코드를 직접 작성하는 능력보다, AI를 활용해 원하는 결과물을 만들어 내는 능력이 중요해지고 있다는 점이다. 사람이 모든 코드를 처음부터 하나하나 작성하는 것은 비효율적인 경우가 많고, AI를 활용하면 훨씬 빠르게 프로그램의 기본 구조를 만들 수 있다. 그러나 동시에 AI가 만든 결과물이 처음부터 완성도 높게 나오지는 않았다. 실제 사용자가 보기 편한지, 목적에 맞는지, 기능이 제대로 작동하는지는 계속 사람이 확인하고 수정해야 했다. 결국 좋은 결과물을 만들기 위해서는 AI에게 막연히 요청하는 것이 아니라, 문제점과 수정 방향을 구체적으로 설명하며 여러 번 다듬는 과정이 필요했다. 이번 활동을 통해 누구나 프로그램을 개발할 수 있는 시대가 되었지만, 좋은 프로그램을 만들기 위해서는 여전히 인간의 문제 정의 능력과 판단력이 중요하다는 것을 느꼈다. 앞으로도 여러 AI의 장단점을 비교하면서, 어떤 AI가 어떤 유형의 프로그램 제작에 적합한지 더 경험해 보고 싶다.